JAVA BACKEND DEVELOPER

안녕하세요,
김승민입니다.

화면부터 서버, 데이터까지 연결하는
자바 백엔드 개발자입니다.

사용자의 흐름을 이해하고, 예외 상황까지 살핍니다.

프로젝트 보기 >

github.com/peppyksm

SELECTED WORK / 2026

peppyksm@gmail.com

01 / SELECTED WORK

Projects.

직접 맡아 구현한 기능과 설계 과정입니다.

PROJECT 03

provit.

2026.09.14 - 2026.10.08 / 4인 팀

서류부터 면접까지 연결하는 AI 취업 준비 플랫폼

팀장 · 취업문서 / AI 첨삭 / 모의면접

평가 생성 후 저장 실패 시,
기존 평가를 보관해 저장만 재시도합니다.

Java 17 | Spring MVC | React 19 | Oracle 19c

PROJECT 02

frade.

2026.08.20 - 2026.09.08 / 5인 팀

모의 투자와 의견 공유를 잇는 주식 트레이딩 플랫폼

거래기록 · 자산 포트폴리오 / 결과 발표

최신순 정렬 후 25건 조회와 자산 계산을 통해
거래 데이터를 사용자의 자산 정보로 연결합니다.

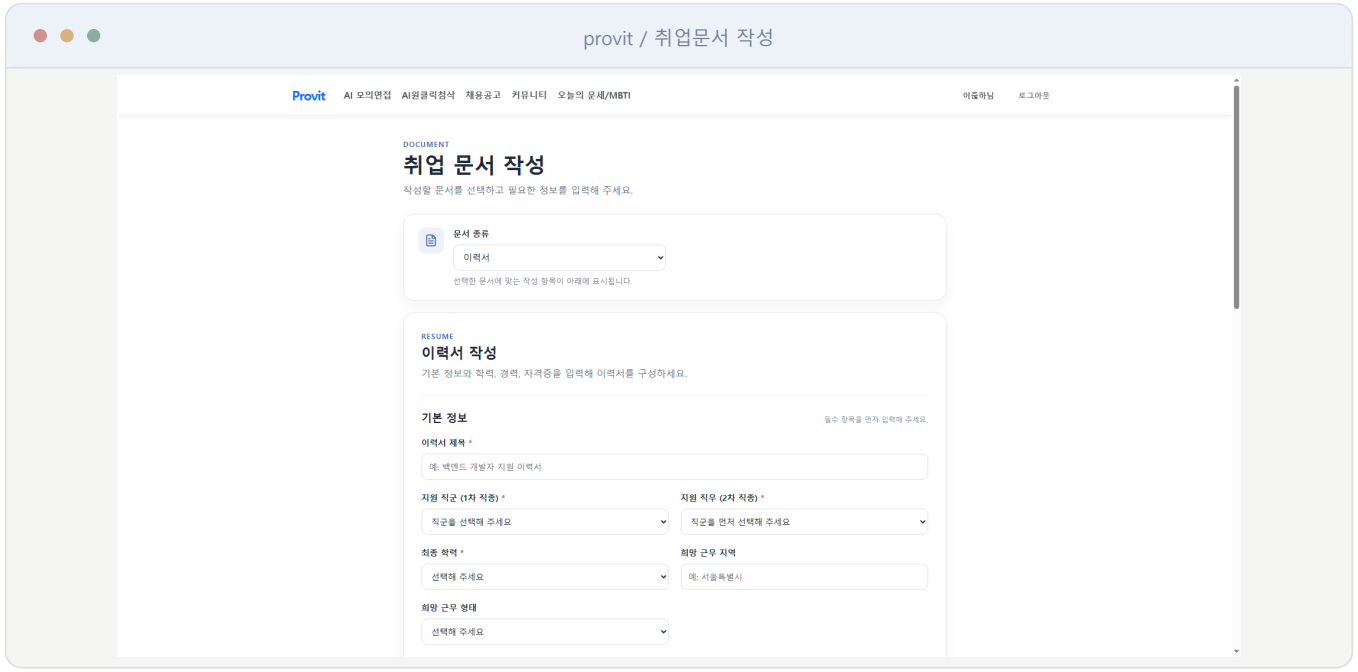
Java 17 | Spring MVC | JSP | Chart.js

3차 프로젝트 / PROJECT 03

provit.

2026.09.14 - 2026.10.08 / 4인 팀

팀장 · 프로젝트 총괄 및 일정 관리



제공된 최종 화면 자료. 개인 데이터는 화면 구성 예시입니다.

서류부터 면접까지 이어지는 취업 준비

취업문서를 작성하고 AI 첨삭으로 보완한 뒤, 자신의 서류를 바탕으로 면접을 연습하는 서비스입니다.

MY CONTRIBUTION

이력서·자소서 CRUD / PDF 등록·조회·삭제
AI 통합 첨삭·기록 / 모의면접 진행·평가·저장

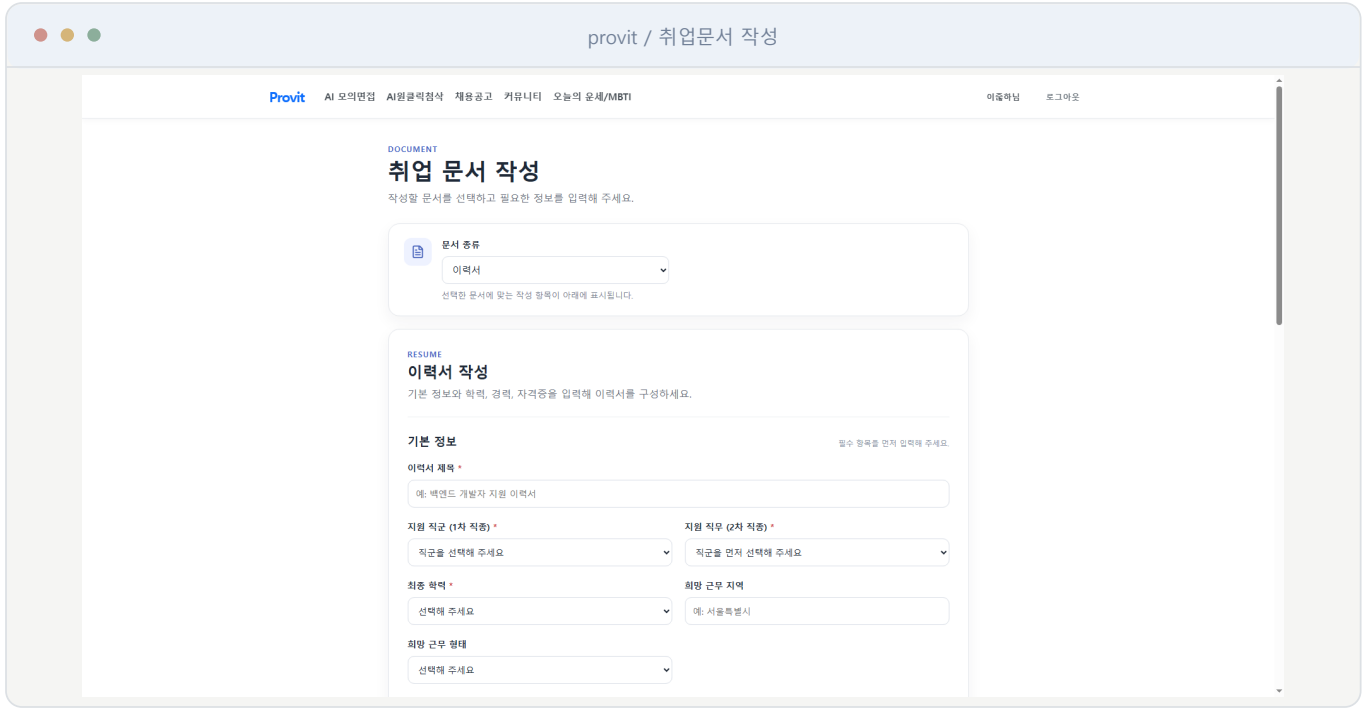
Java 17 / Spring MVC 5.3.39 / Tomcat 9
React 19 / MyBatis / Oracle 19c / OpenAI API

공개 소스 첨삭 기여 PR #53

PROJECT 03 / DOCUMENTS

Documents.

복합 문서와 파일을 함께 다루되, 저장 책임과 실패 경계를 분명히 합니다.



화면: 최종 화면설계 자료 / 구현: DocumentServiceImpl, LocalPortfolioFileStorage

복합 문서는 한 트랜잭션으로

이력서 본문과 학력·경력·자격증을 함께 저장합니다. 이력서·자소서는 CRUD, PDF는 등록·조회·삭제를 제공합니다.

소유권과 입력을 서버에서 검증

필수값·날짜·직무 코드와 문서 소유권을 확인합니다. PDF는 용량·확장자·시그니처를 검사하고 UUID 파일명으로 공개 경로 밖에 저장합니다.

DB 커밋과 파일 정리 시점 분리

롤백 시 새 파일을 정리하고, 기존 파일은 DB 커밋 후 삭제합니다. 삭제 실패는 로그로 남기며 별도 재처리는 없습니다.

PROJECT 03 / AI REVIEW

AI Review.

여러 취업문서를 한 번에 검토하고, 수정 제안과 근거를 함께 전달합니다.



화면 구성용 결과 예시입니다. 실제 선택 서류의 분석 결과나 성과 지표를 뜻하지 않습니다. 의미적 정확성은 별도 품질 확인이 필요합니다.

한 묶음으로 전달하는 문서

이력서·자기소개서와 선택 PDF, 첨삭 조건을 함께 전달합니다. 정상 처리의 생성 호출은 1회입니다.

구조와 근거를 함께 확인

JSON 구조·필수 필드·길이를 검증합니다. 텍스트 인용문은 실제 입력과 대조하고, PDF는 문서 종류와 페이지 범위를 검사합니다.

수정 제안에서 다음 준비까지

종합 의견, 강점·개선점, 수정 전후 비교, 서류 간 일관성 점검과 면접 준비 추천을 제공합니다.

PROJECT 03 / ARCHITECTURE

Request & Storage.

외부 AI 호출과 DB 트랜잭션을 분리하고, 같은 요청의 재전송을 구분합니다.

01 **PROCESSING · 원문 저장**

짧은 DB 트랜잭션

>

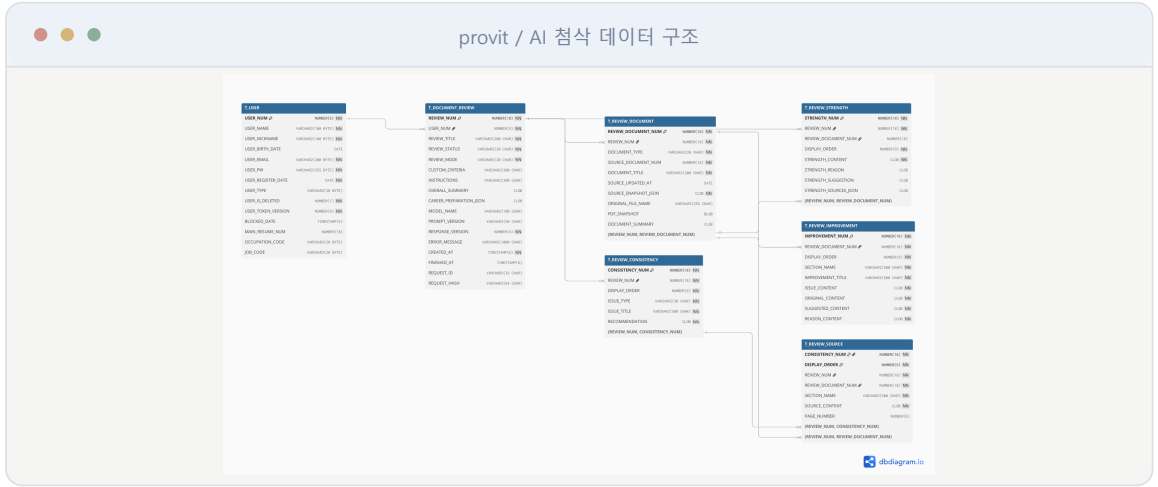
02 **AI 호출 · 응답 검증**

DB 트랜잭션 밖에서 실행

>

03 **첨삭 결과 저장**

별도 DB 트랜잭션



첨삭 서비스 코드

requestId + 문서 ID·조건 해시

동일 사용자·requestId에서 문서 ID와 첨삭 조건의 SHA-256 해시가 같으면 기존 상태·결과를 반환합니다. 조건이 바뀌면 409이며, 새 requestId는 같은 문서도 새 요청입니다.

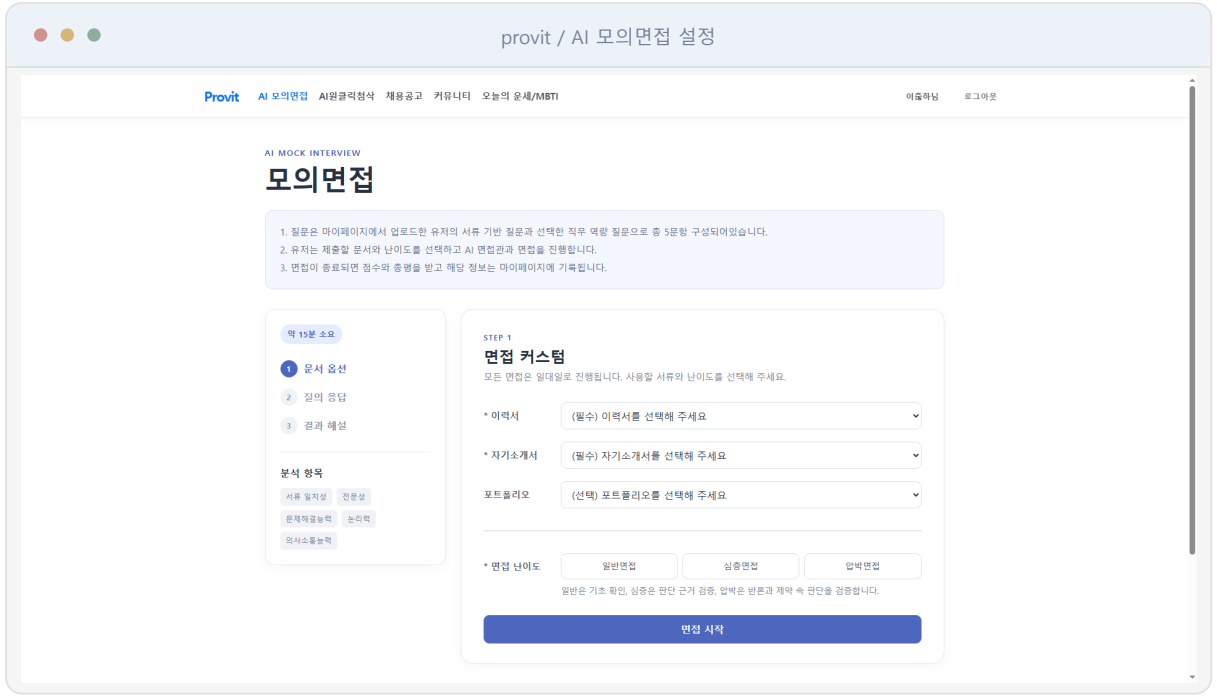
원문 보관과 기록 조회의 범위

당시 JSON·PDF는 DB에 보관합니다. 기록 API는 인용 구절·수정안·결과를 조회합니다. 전체 보관본 열람 API와 잔류 PROCESSING의 자동 복구는 구현하지 않았습니다.

PROJECT 03 / AI INTERVIEW

AI Interview.

5문항의 면접을 진행하며, 정상 완료 흐름의 생성 호출을 4회로 구성했습니다.



이력서·자소서·선택 포트폴리오와 난이도를 설정하는 화면입니다.

01 **면접 시작**

1~3번 질문 일괄 생성

02 **3번 답변 제출**

4번 문제해결 질문 생성

03 **4번 답변 제출**

5번 꼬리질문 생성

04 **5번 답변 제출**

전체 질의·응답 기반 최종 평가

1·2번 답변에는 이미 생성한 다음 질문을 반환합니다. 선택 공고 요약과 사용자 서류를 반영하며, 최종 평가에 시작 시점의 서류·PDF를 다시 전달합니다.

PROJECT 03 / PROBLEM SOLVING

Reliability.

저장에 실패해도, 이미 생성한 평가 결과를 잃지 않도록.



5가지 평가 항목 · 동일 가중 평균

서류 일치성·전문성·문제해결능력·논리력·의사소통능력을 0~100점으로 받습니다. 서버가 범위를 검증하고 평균을 소수점 첫째 자리까지 반올림합니다.

테스트 화면. 점수는 모델의 학습 참고용이며 실제 채용 역량·기술 숙련도 지표가 아닙니다.

문제	최종 평가가 생성된 뒤 DB 저장에 실패하면, 전체 흐름 재실행으로 같은 평가를 다시 생성할 수 있습니다.
확인·검토	AI 생성과 DB 저장은 다른 단계입니다. 일시 오류와 현재 코드의 영구 오류 분류(무결성 위반·ORA-12899)를 구분했습니다.
개선	세션별 동기화로 처리하고, 일시 실패에는 평가·답변을 보관합니다. 같은 마지막 답변을 재제출하면 기존 평가로 저장만 재시도하며, 기록과 평가는 한 트랜잭션으로 저장합니다.
검증	기존 대체 의존성 테스트 기록에서 평가 재사용과 영구 오류 후 반복 방어를 확인했습니다. 이번에 실 DB·AI를 재실행한 결과는 아닙니다.

서버 상태·브라우저 초안을 복원해도 타이머는 유지합니다. 메모리 세션은 재시작·인스턴스 변경 시 복구할 수 없습니다. KST 23:55 만료, 00:00까지 면접 차단.

PROJECT 03 / PROBLEM SOLVING

Text Length.

문자 수 제한을 지켜도 저장되지 않던 평가 문구, 길이 단위부터 확인했습니다.

500 BYTE

기존 평가 컬럼의 최대 길이

750 BYTE

AL32UTF8 완성형 한글 250자

250 CHAR

문자 수 기준의 변경 정의

문제와 원인 확인

일반적인 완성형 한글은 AL32UTF8에서 글자당 3바이트입니다. 평가 250자는 750바이트여서 VARCHAR2(500 BYTE)를 초과했습니다. Java의 500자 허용도 AI의 250자 제한과 달랐습니다.

검토·개선·검증

AI·Java·DB의 길이 기준을 함께 맞추고 마이그레이션을 준비했습니다. 별도 Oracle 테스트 테이블에서 한글·혼합문자·경계값 저장을 검증했고, 기존 데이터가 새 제한을 넘으면 ALTER 전에 중단하도록 했습니다.

면접 필드의 변경 정의

질문	120자	`VARCHAR2(120 CHAR)`
답변	1,000자	`VARCHAR2(1000 CHAR)`
평가	250자	`VARCHAR2(250 CHAR)`

CHAR는 문자 길이 단위입니다. 고정 길이 CHAR 자료형이나 바이트 수와 구분합니다. 앱은 UTF-16 기준이므로 일부 이모지는 2로 계산합니다.

적용 상태 기존 프로젝트 DB에는 마이그레이션·서버 재배포가 필요합니다. 일반 이력서 mainDuty는 운영 DDL 미확인인 조건부 반례로 남아 있습니다.

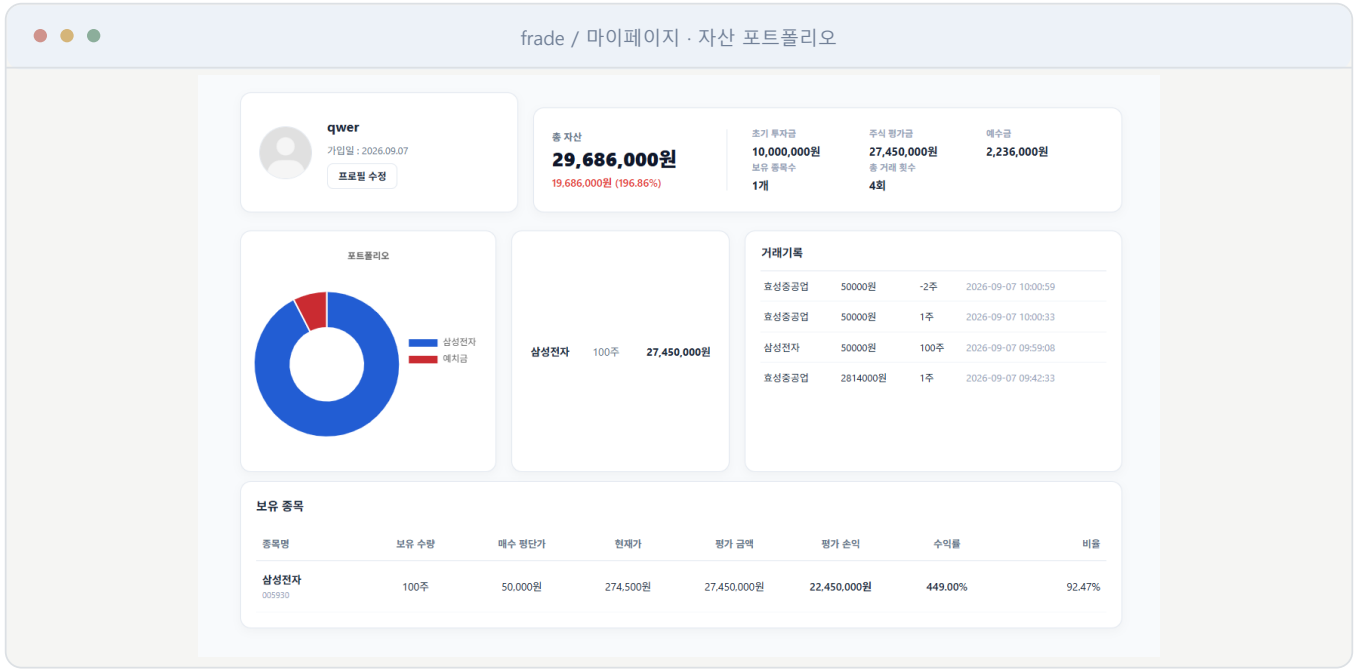
길이 검증 근거

2차 프로젝트 / PROJECT 02

frade.

2026.08.20 - 2026.09.08 / 5인 팀

거래기록 · 자산 포트폴리오 / 결과 발표



최종 화면 자료. 화면의 금액과 수익률은 테스트 데이터입니다.

거래 데이터를 나의 자산 정보로

실제 시세를 바탕으로 모의 투자를 경험하고, 커뮤니티에서 투자 의견을 나누는 플랫폼입니다.

MY CONTRIBUTION

거래기록 저장·조회 DAO/SQL / 마이페이지 자산
ERD 설계 참여·명세 작성 / 결과 발표

Java 17 / Spring MVC 5.3.37 / MyBatis
Oracle / JSP / Chart.js

공개 소스 자산 계산 코드

PROJECT 02 / TRANSACTION HISTORY

Transaction History.

사용자별 체결 이력을 저장하고, 화면에 필요한 최신 거래를 정확한 순서로 가져옵니다.

history_mapper.xml

```
SELECT * FROM (
  SELECT s.stock_name, h.trade_price,
         h.trade_cnt, h.trade_date
  FROM t_history h
  JOIN t_stock s
    ON s.stock_code = h.stock_code
  WHERE h.user_num = #{userNum}
  ORDER BY h.trade_date DESC
) WHERE ROWNUM <= 25
```

담당 거래 조회 SQL

사용자별 조회와 종목명 JOIN

사용자 번호로 조회 범위를 제한하고 T_STOCK과 JOIN해 종목명을 함께 전달합니다. 거래일시 내림차순 정렬 뒤 ROWNUM을 적용합니다.

전체·기간별 조회는 DAO/SQL로

오늘은 00:00부터 다음 날 00:00 미만, 최근 일주일은 6일 전부터, 최근 한 달은 한 달 전부터 조회합니다.

마이페이지는 최근 25건

현재 마이페이지는 최근 거래 25건 조회를 사용합니다. 기간별 SQL 구현과 사용자 화면 필터 완료는 구분하며, 최근 건수를 전체 누적 거래 횟수로 표현하지 않습니다.

구현: HistoryDAO / HistoryDAOImpl / history_mapper.xml

PROJECT 02 / ASSETS & LIVE UPDATES

Asset Portfolio.

조회·계산·표시의 역할을 나누고, 표와 차트가 같은 자산 기준으로 움직이게 합니다.

계산 기준

평균 매수가	매입원가 ÷ 보유수량
평가금액	현재가 × 보유수량
종목 손익	평가금액 - 매입원가
종목 수익률	손익 ÷ 매입원가 × 100
총자산	평가금액 합계 + 예치금
종목 비중	종목 평가금액 ÷ 총자산 × 100

보유수량 0 초과만 조회 / 분모 0은 비율 0 / 비율은 소수 둘째 자리 반올림. 평균 단가는 정수 나눗셈이며 예치금이 없으면 0원으로 처리합니다.

자산 계산 코드

화면에 맞춘 조회 모델

PortfolioInfoDTO에 종목별 계산값을 담고, AssetsInfoDTO와 MyPagePortfolioDTO로 자산 요약·최근 거래·Chart.js 차트 데이터를 전달합니다.

chart-tick으로 평가값 갱신

유효한 양수 가격만 반영해 평가금액·손익·수익률·비중을 다시 계산합니다. 중복 구독을 막고 pagehide에서 닫은 연결은 pageshow에서 다시 구독합니다.

현재 범위와 협업에서 배운 점

초기 투자금은 1,000만 원으로 고정됩니다. 시세만 갱신하며 수량·예치금은 재조회가 필요합니다. 팀의 시세 수집 기능과 연결하며 공통 데이터 형식·계산 기준, 반영 전 최신 브랜치 확인의 중요성을 배웠습니다.

02 / INTRODUCTION

About me.

잘 동작하는 순간부터
실패한 다음까지 생각합니다.

사용자가 어떤 흐름으로 서비스를 이용하는지 먼저 이해하려고 합니다. 화면의 입력이 서버와 데이터에 어떻게 이어지는지 살피고, 각 단계의 책임과 기준을 분명히 하는 것을 중요하게 생각합니다.

오류를 만나면 현재 상태와 원인을 확인하고, 같은 문제가 반복되지 않을 방법을 고민합니다. 정상 동작뿐 아니라 중복 요청, 저장 실패, 다시 시도하는 순간까지 개발의 일부로 봅니다.

이름	김승민 / Kim Seungmin
지원 직무	자바 백엔드 개발자
이메일	peppyksm@gmail.com
GitHub	github.com/peppyksm

WORKING TOGETHER

같은 맥락을 공유하는 협업

진행 상황과 문제를 공유하고, 변경 이유와 확인할 내용을 기록합니다. 팀장으로서 서로의 작업이 연결되는 지점을 미리 맞추려 합니다.

03 / TOOLKIT

Skills.

백엔드를 중심으로 화면 구현과 협업까지 연결하는 기술입니다.

01 Backend

Java | Spring Framework | Tomcat

REST API와 서비스 로직을 구현하고, 입력 검증과 트랜잭션 경계를 관리합니다.

02 Frontend

HTML | CSS | JavaScript | React | JSP

사용자 입력부터 API 연동과 결과 표시까지, 화면과 서버의 흐름을 연결합니다.

03 Database

Oracle | MyBatis | SQL

JOIN 조회와 복합 데이터 저장, 원문 보관과 문자열 길이 기준을 다룹니다.

04 AI Integration

OpenAI API | JSON Schema

서류 기반 생성 흐름, 응답 검증, 재전송과 저장 실패 처리를 구성합니다.

05 Collaboration

Git | GitHub | Notion

코드와 작업 상황을 공유하고, 요구사항과 결정 내용을 기록합니다.

06 Build Tools

Maven

의존성과 빌드 설정을 관리하고, 실행·검증 가능한 환경을 구성합니다.

05 / GET IN TOUCH

Contact.

좋은 서비스를,
함께 만듭니다.

자바 백엔드 개발자 김승민
프로젝트와 개발에 관한 이야기를 나눠요.

EMAIL

peppyksm@gmail.com

GITHUB

github.com/peppyksm

Provit 소스 첨삭 기여 PR #53 Frade 소스

NEXT STEPS

면접 세션의 공용 저장소 전환 / 실제 서류 기반 질문·첨삭 품질 평가 / 호출 지연·실패율 측정